MXEN3005
Robot Manipulation Project

Final Report

Jack Gilbert
21468919

May 22, 2026

Contents

List of Figures

1 Introduction

This report outlines the package design of a ROS2 package responsible for the operation of a wx250s 6DOF robot arm with a PlayStation 4 (PS4) controller. The package allows for direct joint control, end-effector relative cartesian control and precise joint positioning of the arm as well as visualization using RViz. A fire node is also used for integration with a tower defense minigame.

2 ROS2 Package Design

The ROS2 package was created with 2 launch files, intended to be launched on 2 different computers. The ”Controller” launch file would run on the computer connected to the PS4 controller, launching the node for publishing the controller inputs and for RViz visualization of the robot. The ”Cannon” launch file would be launched on the computer connected to the robot, responsible for all control of the arm.

In Figure 1 a ROS2 node graph of the package can be seen, including all nodes and interfaces used as well as showing which nodes and topics are launched on each computer. The ”wx250s” and ”RViz” seen on the graph were added to improve interpretability as though they are not strictly ROS2 nodes they are interfaced with by the package.

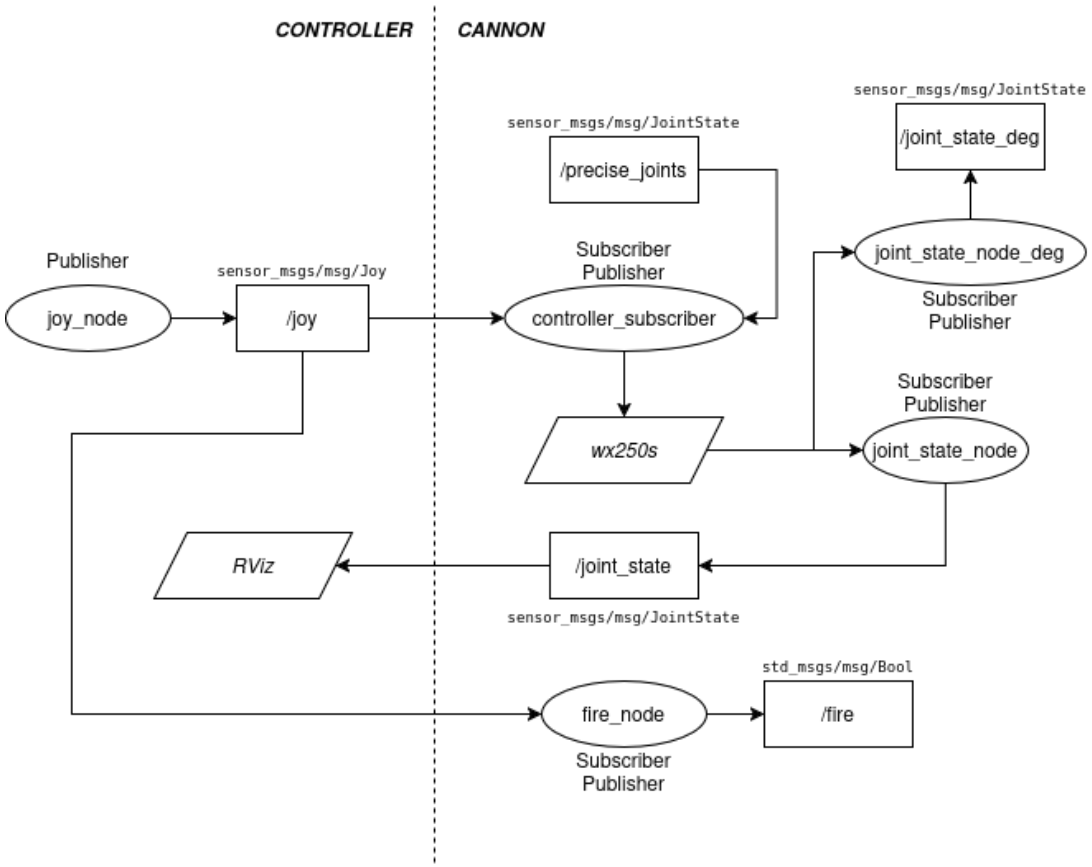

Figure 1: ROS2 Graph

3 Control Algorithms

Our package design implemented one node, the ”controller_subscriber” node, to be used for sending commands to the robot arm. The node is subscribed to 2 topics: /joy and /precise_joints, each with their own dedicated listener callback. The node also includes a timer callback that runs at a rate of 20Hz.

The following sections focus individually on each control algorithm used to achieve control of the wx250s. The node also provides additional functionality for changing between modes using the PS4 controller that is not covered in this report.

3.1 Joint Control

Joint control was achieved through very simple means, mapping the axes each of the 2 joysticks and D-Pad to a specific joint. Figure 2 shows how these inputs were mapped.

The control scheme was split into two callbacks, a listener callback subscribed to the joy topic and a timer callback refreshing at 20Hz. The listener callback updated a joint goal variable based on inputs to the axes in Figure 2 while the timer callback would move the arm to said joint goals. A drift flag was implemented to stop the arm from drifting while no inputs are being made.

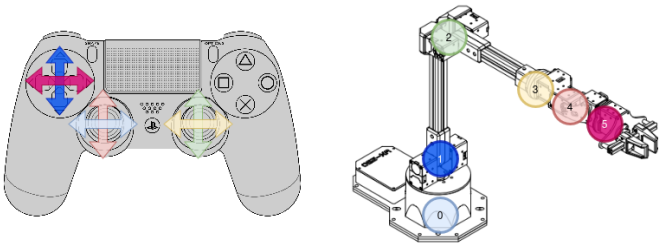

Figure 2: Joint Control Mapping

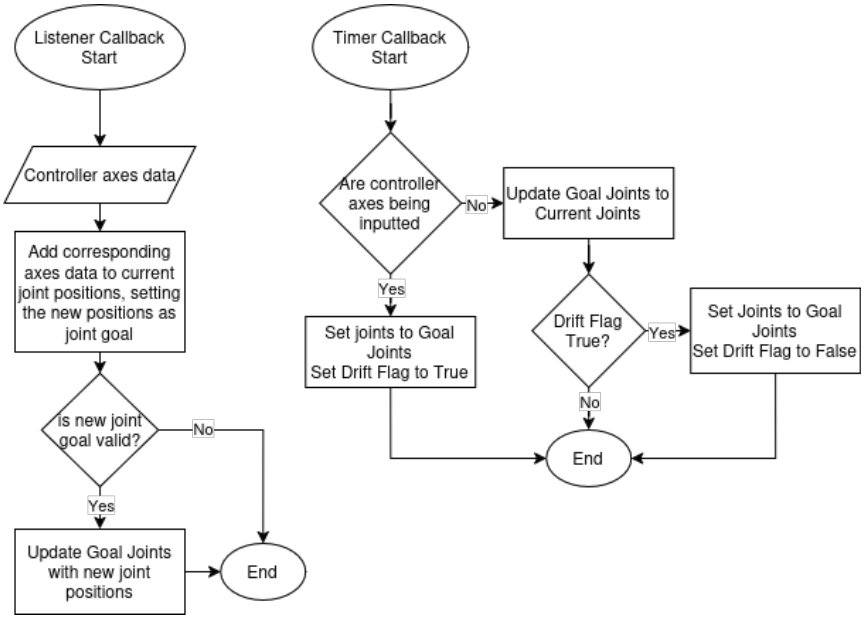

Figure 3: Joint Control Flowchart

3.2 Cartesian Control

Cartesian control was achieved similarly to joint control, utilizing the same timer callback. The controller mapping for this mode is seen in Figure 4.

When cartesian mode is first entered the pose of the end effector is saved (as reference htm) so that its pose is locked during movement. Figure 5 shows the listener callback of cartesian control mode (in the actual code the functionality seen is split across functions however it is combined here for improved readability). The arm is moved to goal joints in the shared timer callback in Figure 3. Intermediate IK frames are mentioned in the flowchart but never occur in practice during cartesian control due to the low speed of the arm in this control mode.

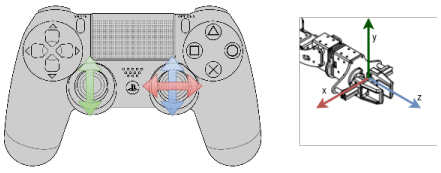

Figure 4: Cartesian Control Mapping

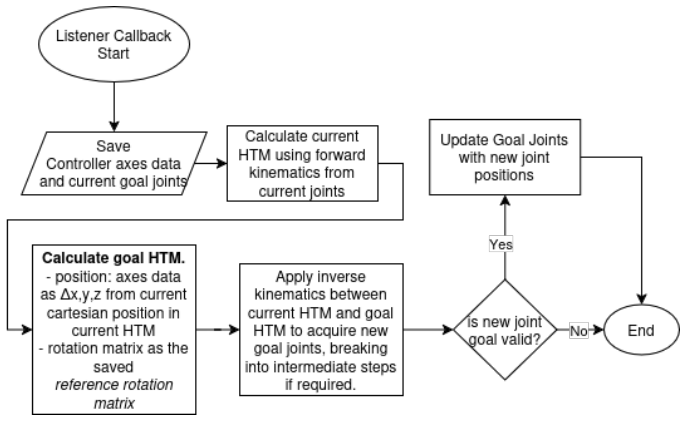

Figure 5: Cartesian Control Flowchart

3.3 Precise Joint Publisher

Precise joint control was implemented as a topic that the controller subscriber was subscribed to in addition to the joy topic, as can be seen in Figure 1. On this topic an array of 6 joint angles can be inputted as ”position”.

```
ros2 topic pub --once /precise_joints sensor_msgs/msg/JointState "{position: [j0, j1, j2, j3, j4, j5]}"
```

There are 2 callbacks used in our precise joints implementation both in the controller subscriber node, ”precise callback” which is subscribed to the /precise_joints topic and a 20Hz timer callback. The logic in each callback can be seen in Figure 6.

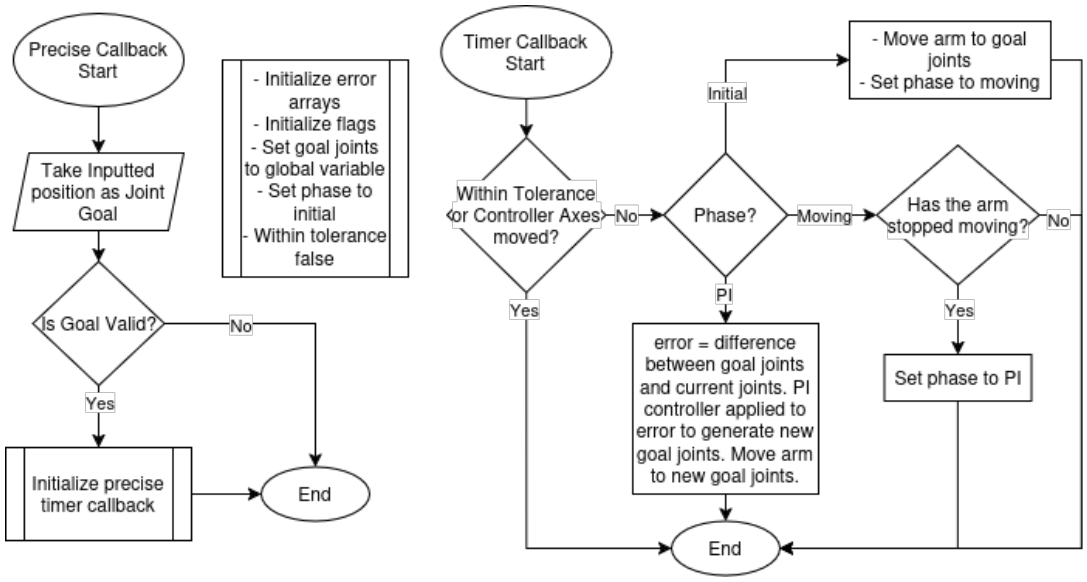

Figure 6: Precise Joints Flowchart

The PI controller was implemented due to the fact that joints are not automatically set precisely, instead sagging slightly due to gravity. The PI controller corrects this error to set joints precisely within encoder tolerance of 0.1 degrees. The PI controller we implemented had gains of $K_p = 0.5, K_i = 0.2$. The precise joints movement can be aborted at any time in the timer callback by moving the controller axes (joysticks or d-pad).